IDFC FIRST Bank – Java Coding Round

Revision pack: 15 questions + Q01 solution

These are 15 problem statements, written in the same style as IDFC's printed assessment sheet. Each one is based on a question IDFC has actually asked: in Rohit's own round, on the shared sample sheet, or in candidates' write-ups on LeetCode, Medium and AmbitionBox.

How the IDFC coding round works

- It's in person, usually on your own laptop. The sheet says **25 minutes, Java**.
- You need **correct output** to advance. A good explanation alone was not enough for at least one candidate.
- After coding, you explain your logic, and they ask about edge cases.
- The evaluation form scores four things:
- **Functionality**: you produce the expected output.
- **Approach**: logical problem solving, good data structures, and splitting the problem into parts.
- **Code quality**: readability and structure.
- **Efficiency**: time and space complexity, and no redundant steps.
- The final decision is one of Select / Reject / Needs Further Evaluation.

How to practise

1. Start a 25-minute timer and read only the question file.
2. Write OOP code: a model class or record, an enum where it fits, a service class, and a `main` that prints the examples.
3. Run every example given and check each line of output matches.
4. Out loud, in 3 minutes: explain your approach, data structures, complexity, edge cases, and the first follow-up.

Questions

#	Question	Type	Where it was seen	Time
01	[Car on an XY plane]	OOP coding	Rohit's round	25 min
02	[Library Fine Summarizer]	OOP + maps	Sample sheet, Medium	25 min
03	[Card Discount Calculation]	Rules engine	Sample sheet	25 min
04	[Highest Value Card]	Parsing + comparison	Sample sheet, LeetCode x2	15 min
05	[Lift `move()`]	OOP / LLD	LeetCode	30 min
06	[Log storage with 1-year retention]	LLD + design	LeetCode	30 min
07	[University Academic Tracking System]	LLD with follow-ups	Medium (AVP round)	40 min
08	[Apartment notice ticker board]	LLD	LeetCode	30 min
09	[Mini Instagram]	LLD + schema + API	LeetCode	40 min
10	[Implement a Max Heap]	DS from scratch	LeetCode	20 min
11	[Find All People With Secret]	Graph / DSU	LeetCode	30 min
12	[First 1 in a sorted 0/1 array]	Binary search	LeetCode	15 min
13	[Banking SQL: JOIN + GROUP BY]	SQL	LeetCode	25 min
14	[ECS on EC2 vs Fargate]	AWS concept	LeetCode	10 min
15	[Implement methods in a given class (Account)]	Online assessment style	LeetCode (OA)	30 min

Some notes on the questions:

- The sample sheet's rules for Q3 contradict its expected output. The question file states the only reading of the rules that gives 130 for 100 and 50.
- The examples in every question were checked with a small script.

Q01. Car Movement on an XY Plane

Duration: 25 minutes · **Language:** Java · **Style:** OOP, with the logic explained afterwards

Problem Statement

A car is placed on an XY grid. It has a position `(x, y)` and faces one of four directions: `N`, `E`, `S` or `W`. It is controlled by a string of commands separated by spaces:

Command	Meaning
F<n>	Move forward n cells in the direction the car is facing
B<n>	Move backward n cells without turning
L	Turn left 90° without moving
R	Turn right 90° without moving

`N` increases `y`, `E` increases `x`, `S` decreases `y` and `W` decreases `x`.

The grid can be unbounded, or bounded with width `W` and height `H`, so valid cells are `0 ≤ x < W` and `0 ≤ y < H`. A bounded grid can also contain **obstacles**. The car moves **one cell at a time**. If the next cell is outside the grid or is an obstacle, the car stays on its last valid cell, the move is reported as `BLOCKED at (x,y)`, and the remaining commands are ignored.

Print the final position, the direction, the number of cells actually travelled, and the blocked cell if there was one.

Expected Output Format

```
(<x>,<y>) <dir> distance=<cells> [BLOCKED at (<bx>,<by>)]
```

Examples

Example 1: unbounded grid

```
start (0,0) N, commands "F2 R F3 L B1"
Output: (3,1) N distance=6
```

F2 moves to (0,2). R turns to face E. F3 moves to (3,2). L turns to face N. B1 moves to (3,1).

Example 2: 5×5 grid with an obstacle at (2,2)

```
start (0,0) E, commands "F2 L F4 R F1"
Output: (2,1) N distance=3 BLOCKED at (2,2)
```

F2 moves to (2,0). L turns to face N. During F4 the car reaches (2,1), and the next cell (2,2) is the obstacle, so it stops. `R F1` is ignored.

Example 3: hits the boundary immediately

```
5×5 grid, start (4,4) N, commands "F1"
```

```
Output: (4,4) N distance=0 BLOCKED at (4,5)
```

Example 4

```
5×5 grid, start (1,1) S, commands "R R F3 L F10"
Output: (0,4) W distance=4 BLOCKED at (-1,4)
```

Expected Design (what the panel looks for)

- `enum Direction { N, E, S, W }` with `left()`, `right()`, `dx()` and `dy()`. Don't use a chain of `if/else` statements for turning.
- `record Position(int x, int y)` with `move(Direction d, int step)`.
- `Grid`: either `UnboundedGrid` or `BoundedGrid(width, height, Set<Position> obstacles)`, with `isFree(Position)`.
- `Car`: holds its position, direction and distance travelled, plus `moveForward(n)`, `moveBackward(n)`, `turnLeft()` and `turnRight()`.
- `Command` interface with `execute(Car)`, implemented by `Forward`, `Backward`, `TurnLeft` and `TurnRight`, and a `CommandParser` that turns the string into a list of commands. This is the Command pattern.
- A custom exception or result type for "blocked".

Constraints

- The command string can be empty, which means the car doesn't move.
- `n` is between 1 and 10^6. On an unbounded grid, use `long` coordinates or explain the overflow risk.
- An invalid command such as `X3` or `F` throws `IllegalArgumentException` with a clear message.
- An obstacle on the starting cell is invalid input.

Edge cases the interviewer will ask about

`B` into an obstacle behind the car · four `L` commands in a row · `F0` · lower-case commands · extra spaces · very large `n` on an unbounded grid. On the last one: can you jump straight to the target, and when must you still step one cell at a time?

Follow-ups

1. Support 8 directions (NE, SE, …) and 45° turns. How much code changes? (Ideally only the enum.)
2. Several cars on the same grid, which must not collide. Who owns the occupancy map?
3. Add other vehicle types: a `Bike` that moves 2 cells per `F1`, or a `Tank` that can't move backward. Would you use inheritance or strategy?
4. `undo()` for the last command. (The Command pattern makes this easy.)
5. Add a command `G x y` that goes to a target cell by the shortest path around obstacles (BFS).

Q01 Solution

Each file is standalone, with its own package and its own `main`. To run them in IntelliJ, mark `src` as a source root.

File	What it answers	OOP idea to name	Diagram
`src/q01/base/CarDemo.java`	**The 25-minute solution** (about 85 lines)	Encapsulation, an enum with behaviour, composition, `toString()` override	01-base
`src/q01/fu1_eight_directions/EightDirectionsDemo.java`	8 directions and 45° turns	Open/closed: only the enum changes	02
`src/q01/fu2_undo/UndoDemo.java`	`undo()` of the last command	**Command pattern**, an interface, a stack (`Deque`)	03
`src/q01/fu3_multiple_cars/MultipleCarsDemo.java`	Several cars that must not collide	Shared state owned by `Grid` (an occupancy map)	04
`src/q01/fu4_vehicle_types/VehicleTypesDemo.java`	Bike (2 cells per unit) and Tank (can't reverse)	**Inheritance and polymorphism**, an abstract class, hook methods	05
`src/q01/fu5_go_to_target/GoToTargetDemo.java`	`G x y`, the shortest path around obstacles	BFS, single responsibility (`PathFinder`)	06

Expected output (all verified)

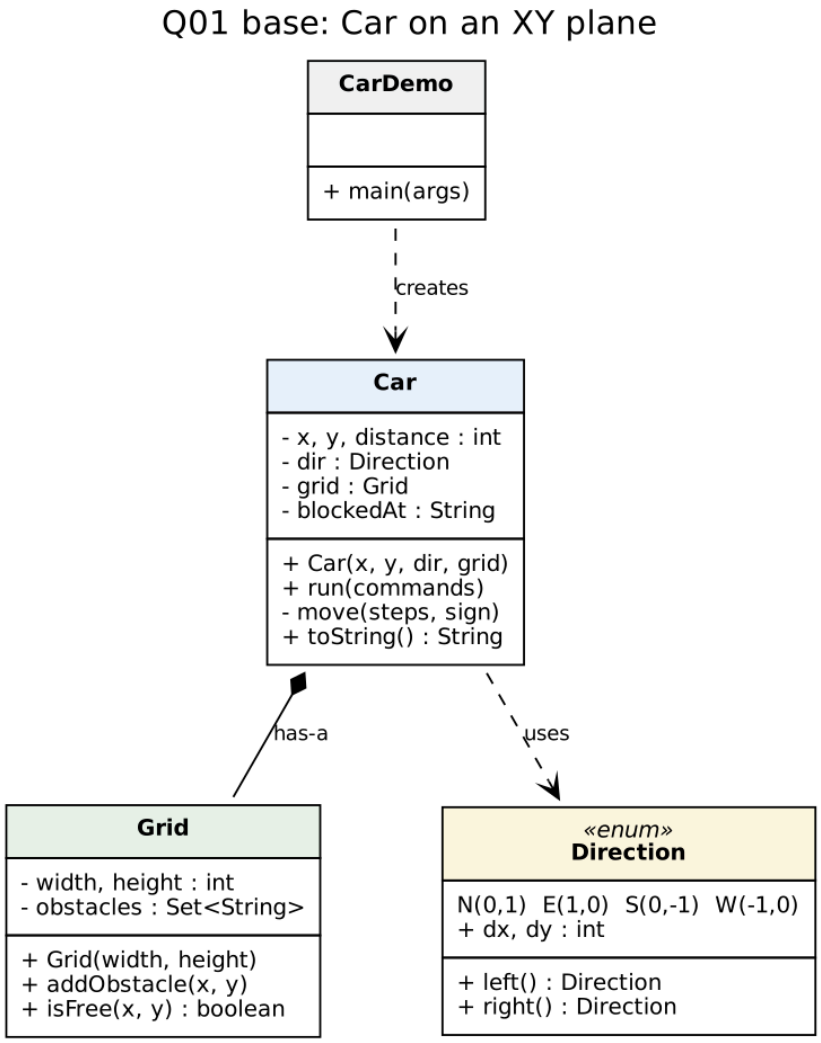
```
base:  (3,1) N distance=6
       (2,1) N distance=3 BLOCKED at (2,2)
       (4,4) N distance=0 BLOCKED at (4,5)
       (0,4) W distance=4 BLOCKED at (-1,4)
fu1:   (3,3) NE distance=3 | (2,6) NW distance=3 | (1,1) NE distance=1 BLOCKED at (2,2)
fu2:   (3,2) E distance=5 -> undo -> (0,2) E distance=8 -> undo -> (0,2) N distance=8
       (0,2) N distance=2 BLOCKED at (0,3) -> undo -> (0,0) N distance=4 -> undo() = false
fu3:   A: (5,2) N distance=7 BLOCKED at (5,3) by car B | B: (5,3) N distance=3
fu4:   Car: (0,2) E distance=4 | Bike: (0,4) E distance=8 | Tank: (1,2) E distance=3 -> Tank
cannot reverse
fu5:   (4,0) S distance=12 | (0,0) N distance=0 BLOCKED at (4,4) unreachable
```

One-line answers

- **8 directions:** list them clockwise in the enum and use `% values().length`. `Car` doesn't change.
- **Undo:** each command implements `execute()` and `undo()` and is pushed onto a stack. A move remembers how many cells it actually moved.
- **Several cars:** `Grid` owns `occupiedBy`, so a car asks it before every step and updates it after.
- **Vehicle types:** an abstract `Vehicle` has a `final run()`. Subclasses override only `cellsPerUnit()` and `canReverse()`.
- **Go to (x, y):** BFS from the car, storing each cell's parent. The first time the target is reached is the shortest path. Rebuild the path backwards from the target.
- **Complexity:**
- Base: O(total steps).
- BFS: O(W×H) time and space.

Diagrams are in `diagrams/` as PNG for viewing and SVG for the PDF. `tools/make.py` regenerates them (requires Graphviz).

Q01 base solution: class diagram



Q01 base solution: code (25-minute version)

```
package q01.base;

import java.util.*;

// Direction the car faces. Each constant carries how one step changes x and y.
enum Direction {
    N(0, 1), E(1, 0), S(0, -1), W(-1, 0);   // order is clockwise: N -> E -> S -> W

    final int dx, dy;                        // change in x and y for one forward step
    Direction(int dx, int dy) { this.dx = dx; this.dy = dy; }

    // Left = one step anticlockwise in the list (+3 is the same as -1 mod 4, avoids negatives)
    Direction left()  { return values()[(ordinal() + 3) % 4]; }
    // Right = one step clockwise in the list
    Direction right() { return values()[(ordinal() + 1) % 4]; }
}

// The plane the car drives on: knows its size and which cells are blocked.
class Grid {
    private final int width, height;                    // width <= 0 means no boundary (unbounded
plane)
    private final Set<String> obstacles = new HashSet<>(); // blocked cells stored as "x,y" for
```

```
O(1) lookup

    Grid(int width, int height) { this.width = width; this.height = height; }

    // Mark a cell as blocked
    void addObstacle(int x, int y) { obstacles.add(x + "," + y); }

    // A cell is free if it is inside the boundary (when there is one) and not an obstacle
    boolean isFree(int x, int y) {
        boolean inside = width <= 0 || (x >= 0 && x < width && y >= 0 && y < height);
        return inside && !obstacles.contains(x + "," + y);
    }
}

// The car: holds its own state and moves itself on a grid.
class Car {
    private int x, y, distance;   // current position and total cells actually travelled
    private Direction dir;        // direction the car is facing
    private final Grid grid;      // grid it drives on (asked before every step)
    private String blockedAt;     // cell that stopped the car; null while not blocked

    Car(int x, int y, Direction dir, Grid grid) {
        this.x = x; this.y = y; this.dir = dir; this.grid = grid;
    }

    // Reads commands like "F2 R F3 L B1" and executes them one by one
    void run(String commands) {
        for (String cmd : commands.trim().split("\\s+")) {   // split on one or more spaces
            if (cmd.isEmpty()) continue;                     // empty input -> nothing to do
            char c = Character.toUpperCase(cmd.charAt(0));   // accept lower case too
            if (c == 'L') dir = dir.left();                  // turn only, no movement
            else if (c == 'R') dir = dir.right();
            else if (c == 'F') move(Integer.parseInt(cmd.substring(1)), 1);   // forward n cells
            else if (c == 'B') move(Integer.parseInt(cmd.substring(1)), -1);  // backward n cells,
same facing
            else throw new IllegalArgumentException("Invalid command: " + cmd);
            if (blockedAt != null) break;                    // rule: once blocked, ignore the
remaining commands
        }
    }

    // Moves one cell at a time so the car stops on the last free cell before a block
    private void move(int steps, int sign) {
        for (int i = 0; i < steps; i++) {
            int nx = x + sign * dir.dx, ny = y + sign * dir.dy;   // next cell (sign -1 reverses
for B)
            if (!grid.isFree(nx, ny)) { blockedAt = "(" + nx + "," + ny + ")"; return; }
            x = nx; y = ny; distance++;                           // step is valid -> commit it
        }
    }

    // Output format: (x,y) DIR distance=n [BLOCKED at (bx,by)]
    @Override public String toString() {
        return "(" + x + "," + y + ") " + dir + " distance=" + distance
                + (blockedAt != null ? " BLOCKED at " + blockedAt : "");
    }
}

// Entry point: builds grids and cars for the 4 examples and prints the results.
public class CarDemo {
    public static void main(String[] args) {
        // Example 1: unbounded grid -> (3,1) N distance=6
        Car c1 = new Car(0, 0, Direction.N, new Grid(0, 0));
        c1.run("F2 R F3 L B1");
        System.out.println(c1);

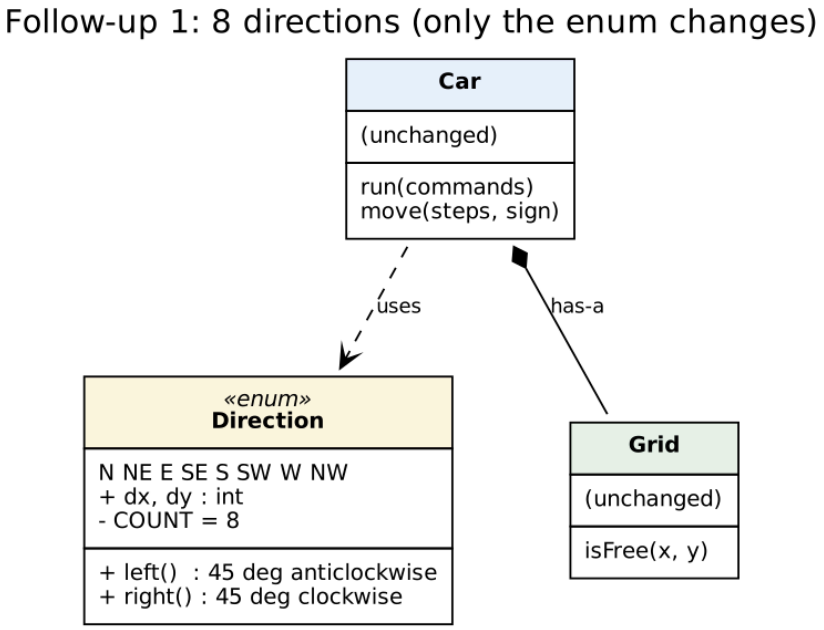
        // Example 2: 5x5 grid, obstacle at (2,2) -> (2,1) N distance=3 BLOCKED at (2,2)
        Grid g = new Grid(5, 5);
        g.addObstacle(2, 2);
```

```
        Car c2 = new Car(0, 0, Direction.E, g);
        c2.run("F2 L F4 R F1");
        System.out.println(c2);

        // Example 3: hits the top boundary straight away -> (4,4) N distance=0 BLOCKED at (4,5)
        Car c3 = new Car(4, 4, Direction.N, new Grid(5, 5));
        c3.run("F1");
        System.out.println(c3);

        // Example 4: two right turns then left boundary -> (0,4) W distance=4 BLOCKED at (-1,4)
        Car c4 = new Car(1, 1, Direction.S, new Grid(5, 5));
        c4.run("R R F3 L F10");
        System.out.println(c4);
    }
}
```

Q01 Follow-up 1: 8 directions




```
package q01.fu1_eight_directions;

import java.util.*;

// FOLLOW-UP 1: "Support 8 directions and 45-degree turns. How much code changes?"
// Answer: only the enum. Car and Grid are the same as the base version,
// because Car only ever asks the enum for dx/dy, left() and right().

// 8 directions in clockwise order. Each step of left()/right() is now 45 degrees.
enum Direction {
    N(0, 1), NE(1, 1), E(1, 0), SE(1, -1), S(0, -1), SW(-1, -1), W(-1, 0), NW(-1, 1);

    final int dx, dy;                          // a diagonal step changes both x and y
    Direction(int dx, int dy) { this.dx = dx; this.dy = dy; }

    private static final int COUNT = values().length;   // 8 - no hard-coded "% 4" any more

    Direction left()  { return values()[(ordinal() + COUNT - 1) % COUNT]; }  // 45 deg
anticlockwise
    Direction right() { return values()[(ordinal() + 1) % COUNT]; }          // 45 deg clockwise
}

// Same as base: size + obstacles
class Grid {
    private final int width, height;
    private final Set<String> obstacles = new HashSet<>();

    Grid(int width, int height) { this.width = width; this.height = height; }

    void addObstacle(int x, int y) { obstacles.add(x + "," + y); }

    boolean isFree(int x, int y) {
        boolean inside = width <= 0 || (x >= 0 && x < width && y >= 0 && y < height);
        return inside && !obstacles.contains(x + "," + y);
    }
}

// Same as base. L/R now mean 45 degrees because the enum changed, not the car.
class Car {
```

```
    private int x, y, distance;
    private Direction dir;
    private final Grid grid;
    private String blockedAt;

    Car(int x, int y, Direction dir, Grid grid) {
        this.x = x; this.y = y; this.dir = dir; this.grid = grid;
    }

    void run(String commands) {
        for (String cmd : commands.trim().split("\\s+")) {
            if (cmd.isEmpty()) continue;
            char c = Character.toUpperCase(cmd.charAt(0));
            if (c == 'L') dir = dir.left();
            else if (c == 'R') dir = dir.right();
            else if (c == 'F') move(Integer.parseInt(cmd.substring(1)), 1);
            else if (c == 'B') move(Integer.parseInt(cmd.substring(1)), -1);
            else throw new IllegalArgumentException("Invalid command: " + cmd);
            if (blockedAt != null) break;
        }
    }

    private void move(int steps, int sign) {
        for (int i = 0; i < steps; i++) {
            int nx = x + sign * dir.dx, ny = y + sign * dir.dy;
            // Talking point: a diagonal step between two obstacles "cuts the corner".
            // If the rules forbid that, also check (x+dx, y) and (x, y+dy) here.
            if (!grid.isFree(nx, ny)) { blockedAt = "(" + nx + "," + ny + ")"; return; }
            x = nx; y = ny; distance++;
        }
    }

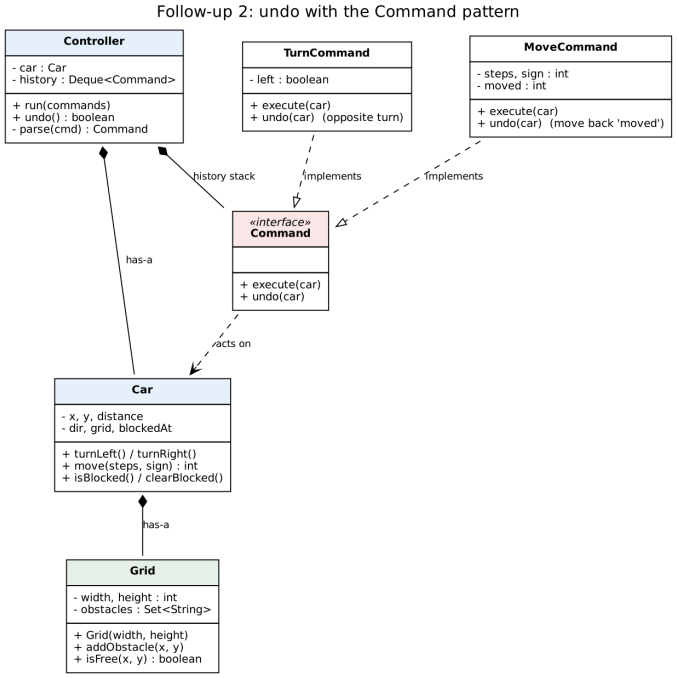
    @Override public String toString() {
        return "(" + x + "," + y + ") " + dir + " distance=" + distance
                + (blockedAt != null ? " BLOCKED at " + blockedAt : "");
    }
}

public class EightDirectionsDemo {
    public static void main(String[] args) {
        // R = 45 deg now: N -> NE, then F3 moves diagonally to (3,3)
        Car c1 = new Car(0, 0, Direction.N, new Grid(0, 0));
        c1.run("R F3");
        System.out.println(c1);              // (3,3) NE distance=3

        // Two lefts = 90 deg: N -> NW -> W
        Car c2 = new Car(5, 5, Direction.N, new Grid(0, 0));
        c2.run("L L F2 R F1");
        System.out.println(c2);              // (2,6) NW distance=3

        // Diagonal into an obstacle
        Grid g = new Grid(5, 5);
        g.addObstacle(2, 2);
        Car c3 = new Car(0, 0, Direction.NE, g);
        c3.run("F4");
        System.out.println(c3);              // (1,1) NE distance=1 BLOCKED at (2,2)
    }
}
```

Q01 Follow-up 2: undo (Command pattern)




```
package q01.fu2_undo;

import java.util.*;

// FOLLOW-UP 2: "Add undo() for the last command."
// Idea: turn every command into an object (Command pattern) that knows how to
// execute AND how to reverse itself. Keep executed commands on a stack.

enum Direction {
    N(0, 1), E(1, 0), S(0, -1), W(-1, 0);

    final int dx, dy;
    Direction(int dx, int dy) { this.dx = dx; this.dy = dy; }

    Direction left()  { return values()[(ordinal() + 3) % 4]; }
    Direction right() { return values()[(ordinal() + 1) % 4]; }
}

class Grid {
    private final int width, height;
    private final Set<String> obstacles = new HashSet<>();

    Grid(int width, int height) { this.width = width; this.height = height; }

    void addObstacle(int x, int y) { obstacles.add(x + "," + y); }

    boolean isFree(int x, int y) {
        boolean inside = width <= 0 || (x >= 0 && x < width && y >= 0 && y < height);
        return inside && !obstacles.contains(x + "," + y);
    }
}

// Every command can do and undo itself
interface Command {
    void execute(Car car);
    void undo(Car car);
}

// Turn: undo is simply the opposite turn
```

```
class TurnCommand implements Command {
    private final boolean left;
    TurnCommand(boolean left) { this.left = left; }

    public void execute(Car car) { if (left) car.turnLeft(); else car.turnRight(); }
    public void undo(Car car)    { if (left) car.turnRight(); else car.turnLeft(); }
}

// Move: remembers how many cells it REALLY moved (it may have been blocked early),
// so undo goes back exactly that many cells, not the requested number.
class MoveCommand implements Command {
    private final int steps, sign;     // sign +1 = forward, -1 = backward
    private int moved;                 // filled in by execute()

    MoveCommand(int steps, int sign) { this.steps = steps; this.sign = sign; }

    public void execute(Car car) { moved = car.move(steps, sign); }
    public void undo(Car car)    { car.move(moved, -sign); }   // cells we came from are always
free
}

// Car exposes small actions; it no longer parses text itself
class Car {
    private int x, y, distance;
    private Direction dir;
    private final Grid grid;
    private String blockedAt;

    Car(int x, int y, Direction dir, Grid grid) {
        this.x = x; this.y = y; this.dir = dir; this.grid = grid;
    }

    void turnLeft()  { dir = dir.left(); }
    void turnRight() { dir = dir.right(); }
    boolean isBlocked() { return blockedAt != null; }
    void clearBlocked() { blockedAt = null; }

    // Returns the number of cells actually moved (needed for undo)
    int move(int steps, int sign) {
        int done = 0;
        for (int i = 0; i < steps; i++) {
            int nx = x + sign * dir.dx, ny = y + sign * dir.dy;
            if (!grid.isFree(nx, ny)) { blockedAt = "(" + nx + "," + ny + ")"; break; }
            x = nx; y = ny; done++;
        }
        distance += done;
        return done;
    }

    @Override public String toString() {
        return "(" + x + "," + y + ") " + dir + " distance=" + distance
                + (blockedAt != null ? " BLOCKED at " + blockedAt : "");
    }
}

// Runs commands and keeps history for undo
class Controller {
    private final Car car;
    private final Deque<Command> history = new ArrayDeque<>();   // stack: last command on top

    Controller(Car car) { this.car = car; }

    void run(String commands) {
        for (String cmd : commands.trim().split("\\s+")) {
            if (cmd.isEmpty()) continue;
            Command c = parse(cmd);
            c.execute(car);
            history.push(c);                    // remember it so it can be undone
            if (car.isBlocked()) break;
```

```
        }
    }

    // Reverses the most recent command; returns false if there is nothing to undo
    boolean undo() {
        if (history.isEmpty()) return false;
        history.pop().undo(car);
        car.clearBlocked();                    // after undo the car is no longer stuck
        return true;
    }

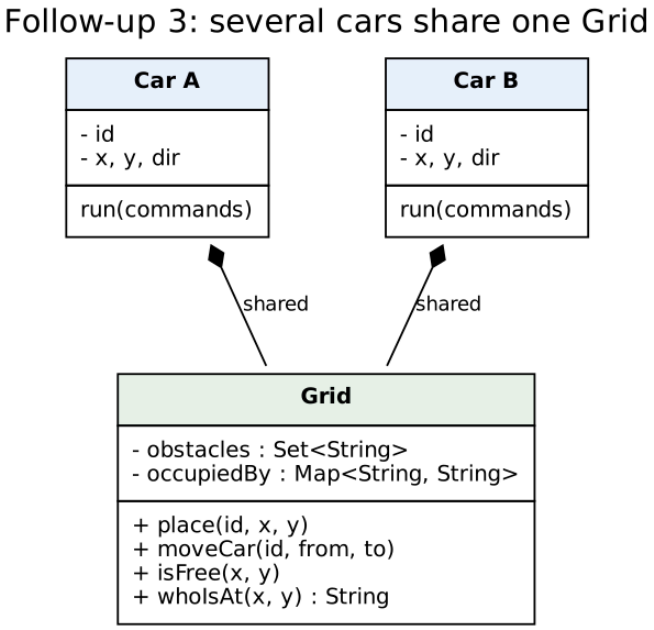
    private Command parse(String cmd) {
        char c = Character.toUpperCase(cmd.charAt(0));
        if (c == 'L') return new TurnCommand(true);
        if (c == 'R') return new TurnCommand(false);
        if (c == 'F') return new MoveCommand(Integer.parseInt(cmd.substring(1)), 1);
        if (c == 'B') return new MoveCommand(Integer.parseInt(cmd.substring(1)), -1);
        throw new IllegalArgumentException("Invalid command: " + cmd);
    }

    Car car() { return car; }
}

public class UndoDemo {
    public static void main(String[] args) {
        Controller ctl = new Controller(new Car(0, 0, Direction.N, new Grid(0, 0)));
        ctl.run("F2 R F3");
        System.out.println(ctl.car());   // (3,2) E distance=5
        ctl.undo();                      // undo F3
        System.out.println(ctl.car());   // (0,2) E distance=8 (distance counts every cell driven)
        ctl.undo();                      // undo R
        System.out.println(ctl.car());   // (0,2) N distance=8

        // Undo after a block goes back only the cells really moved
        Grid g = new Grid(5, 5);
        g.addObstacle(0, 3);
        Controller ctl2 = new Controller(new Car(0, 0, Direction.N, g));
        ctl2.run("F10");
        System.out.println(ctl2.car());  // (0,2) N distance=2 BLOCKED at (0,3)
        ctl2.undo();
        System.out.println(ctl2.car());  // (0,0) N distance=4
        System.out.println(ctl2.undo()); // false - nothing left to undo
    }
}
```

Q01 Follow-up 3: multiple cars




```
package q01.fu3_multiple_cars;

import java.util.*;

// FOLLOW-UP 3: "Several cars on the same grid that must not collide. Who owns the occupancy map?"
// Answer: the Grid. It is the one shared object, so it tracks which cell each car is on.
// A car asks the grid before every step and tells the grid when it moves.

enum Direction {
    N(0, 1), E(1, 0), S(0, -1), W(-1, 0);

    final int dx, dy;
    Direction(int dx, int dy) { this.dx = dx; this.dy = dy; }

    Direction left()  { return values()[(ordinal() + 3) % 4]; }
    Direction right() { return values()[(ordinal() + 1) % 4]; }
}

class Grid {
    private final int width, height;
    private final Set<String> obstacles = new HashSet<>();
    private final Map<String, String> occupiedBy = new HashMap<>();   // "x,y" -> car id

    Grid(int width, int height) { this.width = width; this.height = height; }

    void addObstacle(int x, int y) { obstacles.add(key(x, y)); }

    // Called when a car is placed; two cars cannot start on the same cell
    void place(String carId, int x, int y) {
        if (!isFree(x, y)) throw new IllegalArgumentException("cell " + key(x, y) + " is not
free");
        occupiedBy.put(key(x, y), carId);
    }

    // Moves a car's marker from one cell to the next
    void moveCar(String carId, int fromX, int fromY, int toX, int toY) {
        occupiedBy.remove(key(fromX, fromY));
        occupiedBy.put(key(toX, toY), carId);
    }
```

```
    // Free = inside the boundary, not an obstacle, and no car on it
    boolean isFree(int x, int y) {
        boolean inside = width <= 0 || (x >= 0 && x < width && y >= 0 && y < height);
        String k = key(x, y);
        return inside && !obstacles.contains(k) && !occupiedBy.containsKey(k);
    }

    // Used only for a nicer message: which car is in the way
    String whoIsAt(int x, int y) { return occupiedBy.get(key(x, y)); }

    private static String key(int x, int y) { return x + "," + y; }
}

class Car {
    private final String id;
    private int x, y, distance;
    private Direction dir;
    private final Grid grid;
    private String blockedAt;

    Car(String id, int x, int y, Direction dir, Grid grid) {
        this.id = id; this.x = x; this.y = y; this.dir = dir; this.grid = grid;
        grid.place(id, x, y);                          // register on the shared grid
    }

    void run(String commands) {
        blockedAt = null;                              // each run starts unblocked
        for (String cmd : commands.trim().split("\\s+")) {
            if (cmd.isEmpty()) continue;
            char c = Character.toUpperCase(cmd.charAt(0));
            if (c == 'L') dir = dir.left();
            else if (c == 'R') dir = dir.right();
            else if (c == 'F') move(Integer.parseInt(cmd.substring(1)), 1);
            else if (c == 'B') move(Integer.parseInt(cmd.substring(1)), -1);
            else throw new IllegalArgumentException("Invalid command: " + cmd);
            if (blockedAt != null) break;
        }
    }

    private void move(int steps, int sign) {
        for (int i = 0; i < steps; i++) {
            int nx = x + sign * dir.dx, ny = y + sign * dir.dy;
            if (!grid.isFree(nx, ny)) {
                String other = grid.whoIsAt(nx, ny);
                blockedAt = "(" + nx + "," + ny + ")" + (other != null ? " by car " + other : "");
                return;
            }
            grid.moveCar(id, x, y, nx, ny);            // keep the shared map up to date
            x = nx; y = ny; distance++;
        }
    }

    @Override public String toString() {
        return id + ": (" + x + "," + y + ") " + dir + " distance=" + distance
                + (blockedAt != null ? " BLOCKED at " + blockedAt : "");
    }
}

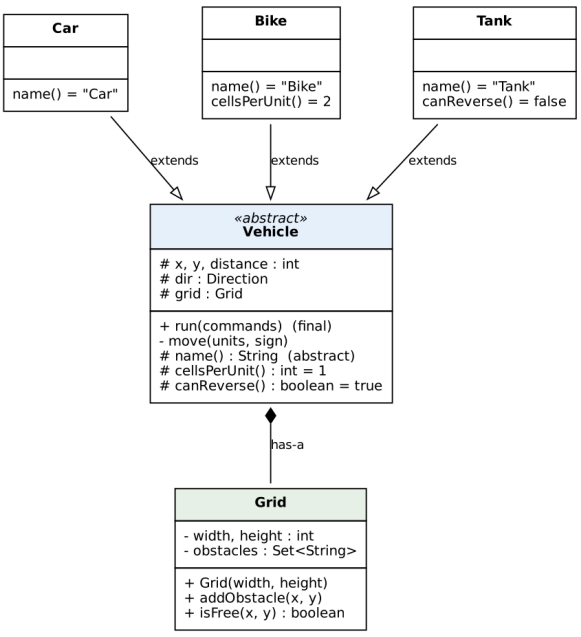
public class MultipleCarsDemo {
    public static void main(String[] args) {
        Grid g = new Grid(10, 10);
        Car a = new Car("A", 0, 0, Direction.E, g);
        Car b = new Car("B", 5, 0, Direction.N, g);

        // Cars take turns. A drives towards B's start cell, but B has already left it.
        b.run("F3");                    // B: (5,3)
        a.run("F5 L F5");               // A: (5,0) then up the same column, stops under B
        System.out.println(a);          // A: (5,2) N distance=7 BLOCKED at (5,3) by car B
```

```
        System.out.println(b);           // B: (5,3) N distance=3
    }
}
```

Q01 Follow-up 4: vehicle types (inheritance)

Follow-up 4: vehicle types with inheritance




```
package q01.fu4_vehicle_types;

import java.util.*;

// FOLLOW-UP 4: "Add other vehicle types: a Bike moves 2 cells per F1, a Tank cannot reverse."
// Answer: INHERITANCE + POLYMORPHISM. An abstract Vehicle holds all the shared logic
// (parsing, stepping, blocking). Subclasses only override the small rules that differ.

enum Direction {
    N(0, 1), E(1, 0), S(0, -1), W(-1, 0);

    final int dx, dy;
    Direction(int dx, int dy) { this.dx = dx; this.dy = dy; }

    Direction left()  { return values()[(ordinal() + 3) % 4]; }
    Direction right() { return values()[(ordinal() + 1) % 4]; }
}

class Grid {
    private final int width, height;
    private final Set<String> obstacles = new HashSet<>();

    Grid(int width, int height) { this.width = width; this.height = height; }

    void addObstacle(int x, int y) { obstacles.add(x + "," + y); }

    boolean isFree(int x, int y) {
        boolean inside = width <= 0 || (x >= 0 && x < width && y >= 0 && y < height);
        return inside && !obstacles.contains(x + "," + y);
    }
}

// Base class: the common behaviour of every vehicle
abstract class Vehicle {
    protected int x, y, distance;          // protected: subclasses may read them
    protected Direction dir;
    protected final Grid grid;
    protected String blockedAt;
```

```
    Vehicle(int x, int y, Direction dir, Grid grid) {
        this.x = x; this.y = y; this.dir = dir; this.grid = grid;
    }

    // Hooks that subclasses override (the only places they differ)
    abstract String name();
    int cellsPerUnit()   { return 1; }     // default: F1 = 1 cell
    boolean canReverse() { return true; }  // default: B is allowed

    // Same parsing loop as the base question, written once for all vehicles
    final void run(String commands) {
        for (String cmd : commands.trim().split("\\s+")) {
            if (cmd.isEmpty()) continue;
            char c = Character.toUpperCase(cmd.charAt(0));
            if (c == 'L') dir = dir.left();
            else if (c == 'R') dir = dir.right();
            else if (c == 'F') move(Integer.parseInt(cmd.substring(1)), 1);
            else if (c == 'B') {
                if (!canReverse()) throw new UnsupportedOperationException(name() + " cannot
reverse");
                move(Integer.parseInt(cmd.substring(1)), -1);
            }
            else throw new IllegalArgumentException("Invalid command: " + cmd);
            if (blockedAt != null) break;
        }
    }

    // n units * cellsPerUnit() cells, still one cell at a time so blocking works
    private void move(int units, int sign) {
        int cells = units * cellsPerUnit();
        for (int i = 0; i < cells; i++) {
            int nx = x + sign * dir.dx, ny = y + sign * dir.dy;
            if (!grid.isFree(nx, ny)) { blockedAt = "(" + nx + "," + ny + ")"; return; }
            x = nx; y = ny; distance++;
        }
    }

    @Override public String toString() {
        return name() + ": (" + x + "," + y + ") " + dir + " distance=" + distance
                + (blockedAt != null ? " BLOCKED at " + blockedAt : "");
    }
}

// Behaves exactly like the original car
class Car extends Vehicle {
    Car(int x, int y, Direction dir, Grid grid) { super(x, y, dir, grid); }
    String name() { return "Car"; }
}

// Faster: every unit is 2 cells
class Bike extends Vehicle {
    Bike(int x, int y, Direction dir, Grid grid) { super(x, y, dir, grid); }
    String name() { return "Bike"; }
    @Override int cellsPerUnit() { return 2; }
}

// Cannot go backwards
class Tank extends Vehicle {
    Tank(int x, int y, Direction dir, Grid grid) { super(x, y, dir, grid); }
    String name() { return "Tank"; }
    @Override boolean canReverse() { return false; }
}

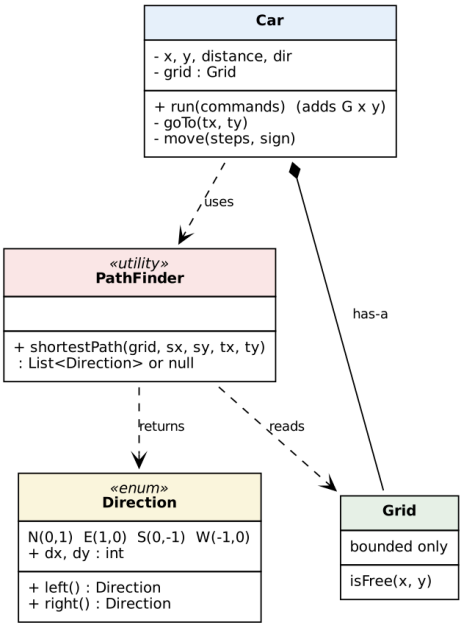
public class VehicleTypesDemo {
    public static void main(String[] args) {
        // One list of the base type -> polymorphism: each object uses its own rules
        List<Vehicle> fleet = List.of(
                new Car(0, 0, Direction.N, new Grid(0, 0)),
                new Bike(0, 0, Direction.N, new Grid(0, 0)),
```

```
                new Tank(0, 0, Direction.N, new Grid(0, 0)));

        for (Vehicle v : fleet) {
            try {
                v.run("F2 R F1 B1");
                System.out.println(v);
            } catch (UnsupportedOperationException e) {
                System.out.println(v + "  -> " + e.getMessage());
            }
        }
        // Car: (0,2) E distance=4
        // Bike: (0,4) E distance=8
        // Tank: (1,2) E distance=3  -> Tank cannot reverse
    }
}
```

Q01 Follow-up 5: go to target (BFS)

Follow-up 5: G x y with BFS shortest path




```
package q01.fu5_go_to_target;

import java.util.*;

// FOLLOW-UP 5: "Add a command G x y: drive to (x,y) by the shortest path around obstacles."
// Answer: BFS (breadth-first search) on the grid. On an unweighted grid, BFS finds
// the shortest path. Store each cell's parent so the path can be rebuilt backwards.
// Needs a bounded grid, otherwise an unreachable target would search forever.

enum Direction {
    N(0, 1), E(1, 0), S(0, -1), W(-1, 0);

    final int dx, dy;
    Direction(int dx, int dy) { this.dx = dx; this.dy = dy; }

    Direction left()  { return values()[(ordinal() + 3) % 4]; }
    Direction right() { return values()[(ordinal() + 1) % 4]; }
}

class Grid {
    private final int width, height;
    private final Set<String> obstacles = new HashSet<>();

    Grid(int width, int height) {
        if (width <= 0 || height <= 0) throw new IllegalArgumentException("BFS needs a bounded
grid");
        this.width = width; this.height = height;
    }

    void addObstacle(int x, int y) { obstacles.add(x + "," + y); }

    boolean isFree(int x, int y) {
        boolean inside = x >= 0 && x < width && y >= 0 && y < height;
        return inside && !obstacles.contains(x + "," + y);
    }
}

// Shortest-path helper, kept separate from Car (single responsibility)
class PathFinder {
```

```
    // Returns the directions to drive, in order; empty if already there; null if unreachable
    static List<Direction> shortestPath(Grid grid, int sx, int sy, int tx, int ty) {
        Map<String, String> parent = new HashMap<>();      // cell -> cell we came from
        Map<String, Direction> stepTo = new HashMap<>();   // cell -> direction used to enter it
        Deque<int[]> queue = new ArrayDeque<>();
        queue.add(new int[]{sx, sy});
        parent.put(sx + "," + sy, null);                   // start has no parent; also marks
visited

        while (!queue.isEmpty()) {
            int[] cur = queue.poll();
            if (cur[0] == tx && cur[1] == ty) break;       // first time we reach target =
shortest
            for (Direction d : Direction.values()) {
                int nx = cur[0] + d.dx, ny = cur[1] + d.dy;
                String k = nx + "," + ny;
                if (grid.isFree(nx, ny) && !parent.containsKey(k)) {
                    parent.put(k, cur[0] + "," + cur[1]);
                    stepTo.put(k, d);
                    queue.add(new int[]{nx, ny});
                }
            }
        }

        String target = tx + "," + ty;
        if (!parent.containsKey(target)) return null;     // never reached

        // Walk back from target to start, then reverse
        LinkedList<Direction> path = new LinkedList<>();
        for (String k = target; parent.get(k) != null; k = parent.get(k))
path.addFirst(stepTo.get(k));
        return path;
    }
}

class Car {
    private int x, y, distance;
    private Direction dir;
    private final Grid grid;
    private String blockedAt;

    Car(int x, int y, Direction dir, Grid grid) {
        this.x = x; this.y = y; this.dir = dir; this.grid = grid;
    }

    // Same as base, plus "G x y" (G and its two numbers are three tokens)
    void run(String commands) {
        String[] t = commands.trim().split("\\s+");
        for (int i = 0; i < t.length; i++) {
            if (t[i].isEmpty()) continue;
            char c = Character.toUpperCase(t[i].charAt(0));
            if (c == 'L') dir = dir.left();
            else if (c == 'R') dir = dir.right();
            else if (c == 'F') move(Integer.parseInt(t[i].substring(1)), 1);
            else if (c == 'B') move(Integer.parseInt(t[i].substring(1)), -1);
            else if (c == 'G') { goTo(Integer.parseInt(t[i + 1]), Integer.parseInt(t[i + 2])); i
+= 2; }
            else throw new IllegalArgumentException("Invalid command: " + t[i]);
            if (blockedAt != null) break;
        }
    }

    // Finds the shortest path, then drives it; the car ends up facing its last move
    private void goTo(int tx, int ty) {
        List<Direction> path = PathFinder.shortestPath(grid, x, y, tx, ty);
        if (path == null) { blockedAt = "(" + tx + "," + ty + ") unreachable"; return; }
        for (Direction d : path) { dir = d; x += d.dx; y += d.dy; distance++; }
    }
```

```
    private void move(int steps, int sign) {
        for (int i = 0; i < steps; i++) {
            int nx = x + sign * dir.dx, ny = y + sign * dir.dy;
            if (!grid.isFree(nx, ny)) { blockedAt = "(" + nx + "," + ny + ")"; return; }
            x = nx; y = ny; distance++;
        }
    }

    @Override public String toString() {
        return "(" + x + "," + y + ") " + dir + " distance=" + distance
                + (blockedAt != null ? " BLOCKED at " + blockedAt : "");
    }
}

public class GoToTargetDemo {
    public static void main(String[] args) {
        // A wall at x=2 for y=0..3 forces the car to go round the top
        Grid g = new Grid(5, 5);
        for (int y = 0; y <= 3; y++) g.addObstacle(2, y);

        Car c = new Car(0, 0, Direction.N, g);
        c.run("G 4 0");
        System.out.println(c);           // (4,0) S distance=12  (up 4, right 4, down 4)

        // Target fully surrounded -> unreachable
        Grid g2 = new Grid(5, 5);
        g2.addObstacle(3, 4); g2.addObstacle(4, 3);
        Car c2 = new Car(0, 0, Direction.N, g2);
        c2.run("G 4 4");
        System.out.println(c2);          // (0,0) N distance=0 BLOCKED at (4,4) unreachable
    }
}
```

Q02. Library Fine Summarizer

Duration: 25 minutes · **Language:** Java · **Source:** IDFC sample sheet

Problem Statement

A community library lends books to people. Sometimes a borrower lends the book on to someone else before it comes back to the library.

You are given an ordered list of lending events. Each event is `(lender, borrower, bookId, daysWithBorrower)`. `"LIB"` means the library.

Rules:

1. A book is allowed out for **10 days in total**, counted from the moment the library lends it until it is returned to `LIB`. This total is the sum of `daysWithBorrower` across the whole chain.
2. The fine is **₹1 for each day beyond 10**.
3. The **person who returns the book to `LIB` pays the whole fine**.
4. If a book hasn't been returned yet, no fine is charged.
5. A book can be lent out again after it is returned. Each lending cycle is fined separately.

For every person who has paid a fine, print the total fine and the books it was collected for.

Expected Output Format

```
<person> : <totalFine> : <bookId>, <bookId>, ...
```

- Print people in the order they **first paid** a fine, and books in the order they were fined.
- Leave out anyone whose total fine is 0.
- If a person is fined for the same book more than once, list the book only once.

Examples

Example 1

```
("LIB","Ace","B1",5), ("Ace","Bob","B1",7), ("Bob","LIB","B1",0)
Output:
Bob : 2 : B1
```

5 + 7 = 12 days, which is 2 days over. Bob returned the book, so Bob pays 2.

Example 2

```
("LIB","Ace","B1",5), ("Ace","Bob","B1",7), ("Bob","LIB","B1",0),
("LIB","Dave","B2",1), ("Dave","Eve","B2",3), ("Eve","LIB","B2",0),
("LIB","Bob","B3",14), ("Bob","LIB","B3",0),
("LIB","Ace","B4",5), ("Ace","Bob","B4",3), ("Bob","Chi","B4",4), ("Chi ","Ace","B4",5),
("LIB","Ace","B5",15),
("Ace ","LIB","B4",0)
Output:
Bob : 6 : B1, B3
Ace : 7 : B4
```

- B1: 12 days, 2 over, Bob pays 2.
- B2: 4 days, no fine.
- B3: 14 days, 4 over, Bob pays 4.
- B4: 5+3+4+5 = 17 days, 7 over, Ace pays 7.
- B5: not returned yet, so no fine yet.

Note: `"Chi "` and `"Ace "` have trailing spaces. You must trim them, or Ace's fine is credited to a different person.

Example 3: the same book in two cycles

```
("LIB","Ace","B1",12), ("Ace","LIB","B1",0), ("LIB","Ace","B1",11), ("Ace","LIB","B1",0),
("LIB","Dan","B2",3), ("Dan","Eve","B2",9)
Output:
Ace : 3 : B1
```

Example 4: exactly 10 days

```
("LIB","Ace","B1",10), ("Ace","LIB","B1",0)
Output: (nothing)
```

Expected Design

- `record LendingEvent(String lender, String borrower, String bookId, int days)`, with names normalised (trimmed) when the record is built.
- `Map<String, Integer> daysOut` for each book in an open cycle. Optionally, `Map<String, String> currentHolder` to validate that the lender actually holds the book.
- `LinkedHashMap<String, FineSummary>` keeps the order in which people first paid. `FineSummary` holds the total and a `LinkedHashSet<String>` of books.
- `FineCalculator`, with `ALLOWED_DAYS = 10` and `FINE_PER_DAY = 1` as constants or configuration. Keep printing separate from calculating.
- Single pass: O(E) time, O(B + P) space.

Edge cases to discuss

A lender who doesn't hold the book (invalid chain: throw or skip?) · an event for a book the library never lent · negative days · a non-zero `days` on the return row · `"lib"` in lower case · an empty input list.

Follow-ups

1. Split the fine between the people who held the book late, not only the returner.
2. Fines with slabs, for example ₹1/day for days 11–20 and ₹2/day after that. Which design pattern fits?
3. Events arrive as a stream and an API needs to show the fines so far at any moment.
4. Show books that are currently overdue and not yet returned, and who is holding each one.

Q03. Card Discount Calculation

Duration: 25 minutes · **Language:** Java · **Source:** IDFC sample sheet

> The rules printed on the sample sheet don't produce its own expected output. Applied in order to 100 and 50, they give 150 → 135 → 115, not **130**. The reading below is the only one that matches the 130. In the real round, **state this ambiguity and ask before you code**. The panel grades that as a positive.

Problem Statement

A customer pays with one or more cards, and each card has a value. Find the **minimum payable amount** after applying the discount rules.

Let `total` be the sum of all card values. Then:

1. Work out each discount the customer is eligible for:
2. **A. 10% off:** `total × 0.90`. Always eligible.
3. **B. Flat ₹20 off:** `total − 20`, never less than 0. Always eligible.
4. **C. 20% off:** `total × 0.80`. Eligible **only if** the minimum card value is ≥ 5 **and** there are **more than 2 cards**.
5. `payable` is the **lowest** of the eligible amounts. Only one of A, B or C is applied; they don't stack.
6. If `payable` is still **more than ₹200**, take off another **₹50**.
7. Return the amount rounded to 2 decimal places, rounding half up.

Function

```
BigDecimal minimumPayable(List<Integer> cardValues)
```

Examples

Input	total	A (10%)	B (−20)	C (20%)	best	> 200?	Output
`[100, 50]`	150	135	130	not eligible (2 cards)	130	no	**130.00**
`[100, 100, 100]`	300	270	280	240	240	yes, −50	**190.00**
`[10, 20]`	30	27	10	not eligible	10	no	**10.00**
`[4, 100, 200]`	304	273.6	284	not eligible (min card 4)	273.6	yes, −50	**223.60**
`[10]`	10	9	0 (floor at 0)	not eligible	0	no	**0.00**
`[90, 90, 90]`	270	243	250	216	216	yes, −50	**166.00**

Expected Design

- Use `BigDecimal` for money, not `double`, and explain why.
- `interface DiscountRule { boolean isEligible(Cart c); BigDecimal apply(BigDecimal total); }`, implemented by `PercentageDiscount`, `FlatDiscount` and `ConditionalPercentageDiscount`. This is the Strategy pattern.
- `DiscountEngine` holds a list of rules, chooses the minimum, then applies a `PostDiscountRule` for the extra ₹50.
- Adding a new rule shouldn't require changing the engine (open/closed principle).

Constraints and validation

- A null or empty list, or any negative value, throws `IllegalArgumentException`.
- Card values are at most 10^6 and there are at most 10^4 cards. Can `int` overflow? Use `long` for the sum.

Edge cases to discuss

Total of exactly 200 (no extra discount) · a total that is 200 or less before the extra rule but more than 200 after a discount, and in which order those rules apply · the flat discount taking the amount below 0 · exactly 2 cards against 3 cards · a card worth exactly 5.

Follow-ups

1. Allow two discounts to stack (10% then ₹20) when a business flag is on.
2. Read the rules from configuration or a database instead of code (rules engine, chain of responsibility).
3. Return an explanation of which rule won (an audit trail), as banks require.
4. Make it concurrent-safe and cacheable for a checkout service.

Q04. Card Game: Highest Value Card

Duration: 15 minutes (candidates reported finishing in about 15) · **Language:** Java · **Source:** IDFC sample sheet; asked onsite, round 1, per two LeetCode posts

Problem Statement

You are given a list of exactly 5 playing cards. Each card is a string in the format `"rank-suit"`:

- rank is one of `2 3 4 5 6 7 8 9 10 J Q K A`
- suit is one of `H` (Hearts), `D` (Diamonds), `C` (Clubs) or `S` (Spades)

Ordering:

- Ranks: `2 < 3 < 4 < 5 < 6 < 7 < 8 < 9 < 10 < J < Q < K < A`
- Suits: `Hearts < Diamonds < Clubs < Spades`
- **Rank is compared first**, then suit. For example, `5-H > 2-S` and `5-S > 5-H`.

Return the card with the highest value. If more than one card has the highest value, which can only happen with duplicate cards, return the **first** one.

Function

```
String highestCard(List<String> cards)
```

Examples

```
highestCard(["5-H", "A-S", "K-D", "10-C", "J-H"])  -> "A-S"
highestCard(["2-H", "3-D", "4-C", "5-S", "6-H"])   -> "6-H"
highestCard(["A-H", "A-D", "K-C", "Q-S", "J-H"])   -> "A-D"   // same rank, D > H
highestCard(["10-H", "J-D", "Q-C", "K-S", "A-H"])  -> "A-H"   // rank beats suit
highestCard(["9-C", "10-H", "10-D", "2-S", "10-H"]) -> "10-D"
```

Expected Design

- `enum Rank` (with a `symbol` and a value given by its order) and `enum Suit`, each with a `fromSymbol(String)` lookup.
- `record Card(Rank rank, Suit suit) implements Comparable<Card>`, or a `Comparator.comparing(Card::rank).thenComparing(Card::suit)`.
- `Card.parse("10-C")`. Split on `-`, not on `charAt(0)`, or `"10"` breaks.
- Make one pass and keep the best card, replacing it only on a **strictly greater** card. That's how "first one wins" works. O(n).

Edge cases to discuss

`"10"` being two characters · lower case such as `"a-s"` · invalid input like `"1-H"`, `"A-X"` or `"AS"` · fewer or more than 5 cards · null elements · why `Collections.max` might not return the first of equal cards (check its contract and say so).

Follow-ups

1. Return the hand sorted from highest to lowest card.
2. Classify the hand: pair, two pair, three of a kind, straight, flush, full house, four of a kind. Use counting maps.
3. Compare two players' hands and return the winner.
4. Let the suit order be configured per game.

Q05. Lift Controller: implement `move()`

Duration: 30 minutes · **Language:** Java · **Source:** IDFC onsite design round, per LeetCode ("Lift class, write move() to tell the sensor up or down")

Problem Statement

Model a single lift in a building with floors `minFloor..maxFloor`. The hardware is reached through a `LiftMotor` interface:

```
interface LiftMotor { void up(); void down(); void openDoor(); }
```

People can add requests at any time with `request(int floor)`. These come from inside the lift or from a floor button.

A controller calls `move()` once per tick. Each call performs **exactly one** action and returns it:

Situation	Action	Returned
There is a pending request for the current floor	Serve it (remove it) and call `openDoor()`	`OPEN`
No pending requests	The lift becomes idle	`IDLE`
Already moving UP and a request is above	Move up one floor (`up()`)	`UP`
Already moving UP and nothing is above	Reverse and move down one floor	`DOWN`
Moving DOWN	Same rules, mirrored	`DOWN` / `UP`
Idle with requests pending	Head toward the **nearest** request. On a tie, go **UP**	`UP` / `DOWN`

This is the **LOOK** algorithm: keep going in the current direction while requests lie ahead, then reverse.

API

```
enum Action { UP, DOWN, OPEN, IDLE }
class Lift {
    Lift(int minFloor, int maxFloor, int startFloor, LiftMotor motor)
    void request(int floor)      // out of range -> IllegalArgumentException; duplicates ignored
    Action move()
    int currentFloor()
}
```

Examples

Example 1

```
Lift at 0 (idle). request(5), request(2)
move() x8 -> UP, UP, OPEN(2), UP, UP, UP, OPEN(5), IDLE
then request(3), request(7)   // at floor 5: both are 2 away, tie -> UP
move() x9 -> UP, UP, OPEN(7), DOWN, DOWN, DOWN, DOWN, OPEN(3), IDLE
currentFloor() == 3
```

Example 2: a request arrives while the lift passes a floor

```
Lift at 4. request(6), request(1)  // 6 is 2 away, 1 is 3 away -> UP
move() -> UP        (now at 5)
request(5)          // added while the lift is at 5
move() x9 -> OPEN(5), UP, OPEN(6), DOWN, DOWN, DOWN, DOWN, DOWN, OPEN(1)
```

Expected Design

- `Direction` enum (`UP`, `DOWN`, `IDLE`) and `Action` enum.
- Pending requests in a `TreeSet<Integer>`, so `higher()` and `lower()` find the next stop in O(log n). Explain why a `List` would be O(n).
- `LiftMotor` is injected, which allows a fake motor in tests (dependency inversion).
- `move()` stays small. Pull out helpers such as `hasRequestsAbove()` and `nearestDirection()`.

Edge cases to discuss

A request for the floor the lift is on while idle (`OPEN` on the next move) · requests at `minFloor` and `maxFloor` · lots of duplicate requests · the lift must never move past the floor limits.

Follow-ups

1. Hall calls with a direction (`call(floor, UP)`): only stop when the call is in the direction of travel.
2. **Several lifts** with a `Dispatcher` that picks a lift (nearest, least loaded). Use the Strategy pattern.
3. Door and lift states (`DOOR_OPEN`, `MOVING`, `MAINTENANCE`) as a **State** pattern.
4. Capacity and weight sensor: skip hall calls when the lift is full.
5. Thread safety: buttons are pressed on other threads while `move()` runs on the controller thread.

Q06. Log Store with 1-Year Retention (no cloud)

Duration: 30 minutes (15 for code, 15 for design talk) · **Language:** Java · **Source:** IDFC onsite design round, per LeetCode ("collect logs, retain 1 year, focus on storage, no cloud services")

Problem Statement

Build the storage layer for an application's logs. Logs must be kept for **365 days** on local disk only: no S3 and no CloudWatch. After 365 days they must be deleted automatically.

Logs are written to **segments**, which stand in for files:

- New entries go into the **active segment**.
- When the active segment reaches `maxEntriesPerSegment`, it is **rolled over**: closed, compressed through a `Compressor`, and archived. A new active segment then starts. In real life the limit is in bytes; entries are used here to keep tests simple.
- `purgeExpired()` deletes every archived segment whose **newest** entry is older than 365 days. Older than 365 days means `now − timestamp > 365 days`. The active segment is removed only if all of its entries have expired.
- `query(from, to, minLevel)` returns the matching entries, oldest first. It must **skip segments** whose time range doesn't overlap `[from, to]`.

API

```
enum Level { DEBUG, INFO, WARN, ERROR }
record LogEntry(Instant timestamp, Level level, String message) {}
interface Compressor { byte[] compress(List<LogEntry> entries); List<LogEntry> decompress(byte[]
data); }

class LogStore {
    LogStore(int maxEntriesPerSegment, Duration retention, Clock clock, Compressor compressor)
    void append(LogEntry e)                      // timestamps are non-decreasing
    int purgeExpired()                           // returns the number of segments deleted
    List<LogEntry> query(Instant from, Instant to, Level minLevel)
    int segmentCount()                           // archived + active
    int segmentsScannedByLastQuery()             // proves the skip works
}
```

Example (time written as "day N")

```
maxEntriesPerSegment = 2, retention = 365 days
append(day1 INFO "start"), append(day2 ERROR "db down"), append(day3 WARN "slow"),
append(day4 INFO "ok"), append(day5 ERROR "timeout")
-> segments: [d1,d2] archived, [d3,d4] archived, [d5] active  -> segmentCount() = 3

query(day3, day5, WARN) -> [day3 WARN "slow", day5 ERROR "timeout"]
segmentsScannedByLastQuery() = 2        // [d1,d2] skipped

clock = day 370
purgeExpired() -> 2                     // [d1,d2] and [d3,d4] are older than 365 days; d5 is
exactly 365 -> kept
query(day1, day5, DEBUG) -> [day5 ERROR "timeout"]
```

Expected Design

- `Segment` holds `minTs`, `maxTs`, `count` and either the entries (active) or compressed bytes (archived).
- `Deque` or `TreeMap<Instant, Segment>` ordered by start time, so purging only removes from the head and queries can binary search to the first overlapping segment.
- `Clock` is injected so tests can control time.
- `append` must be O(1) on average. Compression happens only at rollover.

Design talk (the panel moves on to this)

1. **Sizing:** 2 GB of raw logs a day for 365 days is about 730 GB. Gzip at roughly 8–10× brings that to about 75–90 GB. How much disk would you provision, and with what alert threshold?
2. **File layout:** for example `/logs/app/2026/10/23/app-0001.log.gz`. Roll over by size and by day. Deleting a day's directory is cheap.
3. **Purge job:** a `ScheduledExecutorService` or cron job runs every night. It must be idempotent and safe to run twice.
4. **Write path:** an async appender writes through a bounded queue to a single writer thread. What happens when the queue is full: drop, block, or sample?
5. **Durability:** how often to fsync, and how not to lose the tail of the logs on a crash.
6. **Search:** a per-segment index of min/max timestamp and level counts, and a bloom filter for keywords. When would you move to something like ELK on-prem?
7. Compliance: masking PII and card numbers before storage, because this is a bank.

Q07. University Academic Tracking System

Duration: 40 minutes (the core first, then follow-ups added one by one) · **Language:** Java · **Source:** IDFC AVP round, per Medium ("University Academic Tracking System with 5–6 follow-up conditions")

Problem Statement

Build an in-memory service that tracks students, courses, enrollments and grades.

Core requirements (code these first):

1. `addStudent(id, name)` and `addCourse(code, title, credits, capacity, Set<String> prerequisites)`.
2. `enroll(studentId, courseCode, semester)` returns an `EnrollmentResult` and rejects the enrollment if:
3. the student or course doesn't exist (`NOT_FOUND`)
4. the course is full for that semester (`COURSE_FULL`)
5. a prerequisite hasn't been passed. Pass means any grade except `F` (`PREREQUISITE_NOT_MET`)
6. the student is already enrolled in it this semester, or has already passed it (`ALREADY_ENROLLED`, `ALREADY_PASSED`)
7. the semester's credits would go over **24** (`CREDIT_LIMIT_EXCEEDED`)
8. `recordGrade(studentId, courseCode, semester, Grade)`.
9. `sgpa(studentId, semester)` and `cgpa(studentId)` are credit-weighted and rounded to 2 decimals, half up. If a course is **retaken**, only the **latest** attempt counts in the CGPA.

Grade points: `O=10, A+=9, A=8, B+=7, B=6, C=5, P=4, F=0`.

Example

```
Courses: CS101 (4 credits), MA101 (3 credits), CS201 (4 credits, prereq CS101)

S1, sem 1: CS101 = A+, MA101 = B
  sgpa(S1, 1) = (9*4 + 6*3) / 7 = 54/7 = 7.71

S2 enrolls in CS201 without CS101 -> PREREQUISITE_NOT_MET

S3, sem 1: CS101 = F, MA101 = O       -> sgpa(S3,1) = 30/7 = 4.29
S3, sem 2: retakes CS101 = A          -> sgpa(S3,2) = 32/4 = 8.00
  cgpa(S3) = (8*4 + 10*3) / 7 = 62/7 = 8.86   // the old F is dropped
S3 then enrolls in CS201 in sem 3 -> OK (CS101 is now passed)
```

Expected Design

- Entities: `Student`, `Course`, `Enrollment(studentId, courseCode, semester, Grade grade)`. Use `enum Grade` with `points()` and `isPass()`.
- Repositories: `Map<String, Student>`, `Map<String, Course>`, and enrollments indexed by student and by `(course, semester)` for capacity counts.
- Validation as a list of `EnrollmentRule` objects (`CapacityRule`, `PrerequisiteRule`, `CreditLimitRule`, ...), each returning an optional failure. New rules then plug in without editing `enroll()`.
- `GpaCalculator` is kept separate from storage (single responsibility).

Follow-ups (added one at a time, the way the AVP round did it)

1. **Waitlist:** when a course is full, put the student on a FIFO waitlist. `drop()` frees a seat and automatically enrolls the next student on the waitlist who still passes every rule.
2. **Attendance:** the student must have ≥ 75% attendance to receive a grade; below that the grade is recorded as `F`.
3. **Transcript:** list every attempt, grouped by semester and sorted by course code.
4. **Rank list:** the top `k` students by CGPA for a department. Ties go to more credits earned, then to the student id.
5. **Prerequisite cycles:** reject `addCourse` if it would create a cycle (DFS or topological sort).
6. **Concurrency:** two students enroll for the last seat at the same moment. How do you guarantee only one gets it, in memory and then in SQL? (Row lock, `SELECT … FOR UPDATE`, or optimistic version column.)

Discussion

The DB schema for this (tables, keys, a unique constraint on `(student, course, semester)`), and the SQL for the CGPA computation.

Q08. Apartment Notice Ticker Board

Duration: 30 minutes · **Language:** Java · **Source:** IDFC L2 round, per LeetCode ("notification banner system for an apartment, like a rotating stock-market ticker board; ask clarifying questions")

Problem Statement

An apartment lobby has a display board that rotates through notices. Each call to `next(now)` returns the notice to show in the next slot.

Rules:

1. A notice has `id`, `text`, `priority` (`EMERGENCY > HIGH > NORMAL`) and an active window `[start, end)`. It is **active** when `start ≤ now < end`.
2. If **any EMERGENCY** notice is active, the board rotates **only among the emergency notices**.
3. Otherwise it rotates **round-robin** through every active notice, in the order **priority first, then posting order**.
4. Round-robin continues from the last notice shown. It picks the first active notice that comes after it in the order above, wrapping around to the start. If notices expired or were added in between, still pick the next one in that order.
5. If nothing is active, return `Optional.empty()`. The board shows a default screen.

API

```
enum Priority { EMERGENCY, HIGH, NORMAL }
record Notice(String id, String text, Priority priority, long start, long end) {}
class NoticeBoard {
    void post(Notice n)              // a duplicate id -> IllegalArgumentException
    boolean remove(String id)
    Optional<Notice> next(long now)
}
```

Example

```
post N1 NORMAL "Water cut 10-12"  [0,100)
post N2 HIGH   "Lift maintenance" [0,50)
post N3 NORMAL "Diwali party"     [20,100)

next(10) -> N2      // active: N2, N1 (N3 not started)
next(15) -> N1
next(20) -> N3      // N3 is now active and comes after N1
next(25) -> N2      // wrap around
next(55) -> N1      // N2 expired at 50
next(60) -> N3

post E1 EMERGENCY "Fire drill" [70,80)
next(70) -> E1
next(75) -> E1      // only emergencies while one is active
next(80) -> N1      // E1 expired (end is exclusive)
next(100) -> empty
```

Expected Design

- A comparator on `(priority, postSequence)` and a stored cursor of the last shown key. Don't store an index, because indexes shift when notices expire.
- For many notices, use a `TreeSet<Notice>` with a `higher(lastShown)` lookup, plus a min-heap on `end` for cheap removal of expired notices.
- `Clock` or `now` is passed in so tests can control time.
- Keep the rotation **policy** behind an interface such as `RotationStrategy`, so a weighted rotation can be added later.

Clarifying questions to ask before coding (the panel expects these)

How many screens are there? Is there a fixed duration per slot? Who can post (an admin console)? Are notices scheduled ahead of time? Is it multilingual? Does it need a delivery guarantee if the board is offline?

Follow-ups

1. **Weighted rotation:** HIGH notices appear twice as often as NORMAL ones.
2. Several boards (one per tower), each with its own set of notices plus shared global ones.
3. Push changes to the boards: polling vs SSE/WebSocket vs MQTT. (Relate this to your IoT/MQTT device experience.)
4. Persistence and APIs: `POST /notices`, `GET /boards/{id}/current`, and the DB schema.
5. Thread safety while an admin posts during a rotation.

Q09. Mini Instagram: core LLD + schema + API

Duration: 40 minutes (20 for code, 20 for design) · **Language:** Java · **Source:** IDFC L1 round, per LeetCode ("design basic Instagram: DB choice, API design, schema")

Part A: Code (in memory)

```
class SocialService {
    void createUser(String userId)                 // duplicate -> IllegalArgumentException
    void follow(String follower, String followee)  // self-follow -> IllegalArgumentException;
idempotent
    void unfollow(String follower, String followee)
    long post(String userId, String caption)       // returns a postId; ids increase with time
    void like(String userId, long postId)          // idempotent: liking twice counts once
    int likeCount(long postId)
    List<Long> feed(String userId, int k)          // the latest k posts by the user AND the
people they follow, newest first
}
```

Example

```
createUser A, B, C
follow(A, B)
p1 = post(B, "hello")      p2 = post(C, "sunset")      p3 = post(A, "coffee")
feed(A, 10) -> [p3, p1]
follow(A, C)
feed(A, 10) -> [p3, p2, p1]
unfollow(A, B)
feed(A, 10) -> [p3, p2]
like(A, p2); like(A, p2)
likeCount(p2) -> 1
feed(A, 1) -> [p3]
```

Expected

- `Map<String, Set<String>> following`, `Map<String, Deque<Post>> postsByUser` (newest first), and `Map<Long, Set<String>> likes`.
- Build the feed by a **k-way merge with a max-heap** over each followee's newest posts: O(k log F), not "collect everything and sort".

Part B: Design talk

1. **Schema:** `users`, `follows(follower_id, followee_id, created_at, PK(follower, followee))`, `posts(id, user_id, caption, media_url, created_at)`, `likes(post_id, user_id, PK(post_id, user_id))`. Which indexes?
2. **Database choice:** relational for users and follows, with a wide-column or key-value store for feeds and likes at scale. Be ready to justify it. (One candidate picked NoSQL because transactions weren't needed. Expect pushback.)
3. **APIs:** `POST /posts` (upload media to object storage with a pre-signed URL, then create the post), `GET /feed?cursor=…&limit=20` with cursor paging rather than offset, and `PUT /posts/{id}/like`.
4. **Fan-out on write vs on read**, and the celebrity problem (hybrid: pull posts from celebrities at read time).

5. **Like counter at scale:** async counting through a queue such as AWS SQS, then batch updates to the count.
6. Media: S3 and a CDN, thumbnails generated by a Lambda on upload.

Q10. Implement a Max Heap from Scratch

Duration: 20 minutes · **Language:** Java · **Source:** IDFC L1 round, per LeetCode ("implement max heap")

Problem Statement

Implement a max heap of `int` backed by an array that grows as needed. **Don't use** `PriorityQueue` or any `Collections` sort.

```
class MaxHeap {
    MaxHeap()                           // initial capacity 16, doubles when full
    static MaxHeap heapify(int[] a)     // build in O(n) with bottom-up sift-down; don't modify the
caller's array
    void push(int x)                    // O(log n)
    int pop()                           // remove and return the max; empty ->
NoSuchElementException
    int peek()                          // empty -> NoSuchElementException
    int size()
    boolean isEmpty()
}
```

Examples

```
h = new MaxHeap()
push 5, 3, 8, 1, 9
peek() -> 9
pop() -> 9, pop() -> 8
size() -> 3

heapify([4, 10, 3, 5, 1]) then pop until empty -> 10, 5, 4, 3, 1
heapify([7, 7, 7]) -> pop -> 7, 7, 7           // duplicates
new MaxHeap().pop() -> NoSuchElementException
push 1..100000 then pop all -> strictly descending   // growth plus speed
```

Points to explain

- Index arithmetic: `parent = (i-1)/2`, `left = 2i+1`, `right = 2i+2`.
- Why `heapify` is **O(n)**, not O(n log n): most nodes are near the bottom and only sift down a few levels.
- `siftDown` must choose the **larger** child. Watch the case where only a left child exists.
- Handling `Integer.MIN_VALUE` and `Integer.MAX_VALUE`, which have no overflow risk because values are only compared.

Follow-ups

1. Make it generic: `Heap<T>` taking a `Comparator<T>`. A min heap is then just a reversed comparator.
2. `kthLargest(int[] a, int k)` with a min heap of size k: O(n log k).
3. Heap sort in place.
4. `remove(x)` or `decreaseKey`. Why do you need an index map for O(log n)?
5. A thread-safe version: `synchronized` vs `ReentrantLock` vs `PriorityBlockingQueue`.

Q11. Find All People With the Secret

Duration: 30 minutes · **Language:** Java · **Source:** IDFC Senior Developer coding round, per LeetCode (LeetCode 2092)

Problem Statement

There are `n` people, numbered `0..n-1`. Person `0` knows a secret, and at time `0` tells it to `firstPerson`.

You are given `meetings[i] = [x, y, time]`: persons `x` and `y` meet at `time`. Whenever two people meet and one of them knows the secret, the other learns it too. Sharing happens **instantly**: a person can learn the secret and pass it on in **other meetings at the same time**. Someone who only learns it at a later time can't share it in an earlier meeting.

Return every person who knows the secret after all meetings, sorted in ascending order.

```
List<Integer> findAllPeople(int n, int[][] meetings, int firstPerson)
```

Examples

```
n=6, meetings=[[1,2,5],[2,3,8],[1,5,10]], firstPerson=1        -> [0,1,2,3,5]
  t=0: 0->1;  t=5: 1->2;  t=8: 2->3;  t=10: 1->5

n=4, meetings=[[3,1,3],[1,2,2],[0,3,3]], firstPerson=3         -> [0,1,3]
  t=2: 1 and 2 meet, but neither knows yet;  t=3: 3->1 (and 0-3 again)

n=5, meetings=[[3,4,2],[1,2,1],[2,3,1]], firstPerson=1         -> [0,1,2,3,4]
  t=1: 1->2, then 2->3 at the same time;  t=2: 3->4

n=4, meetings=[[1,2,1],[0,1,2]], firstPerson=3                  -> [0,1,3]
  the meeting at t=1 happens before 1 knows, so 2 never learns it
```

Constraints

$2 \le n \le 10^5$, $1 \le meetings.length \le 10^5$, $1 \le time \le 10^5$, $x \ne y$.

Expected Approach (explain the trade-off)

- Sort the meetings by time and **process each group with the same time together**.
- **Option 1, BFS or DFS per time group:** build a small graph for the group and start from the people in it who already know.
- **Option 2, Union-Find with reset:** union everyone in the group with person 0's set, then **reset** the people in the group who didn't end up connected to 0. This reset is the key insight, and it's the reason the problem is hard.
- O(M log M) for the sort plus about O(M α(n)).

Edge cases

Many meetings at the same time forming a chain · `firstPerson` with no meetings · the same pair meeting at several times · an unordered `meetings` input.

Follow-ups

1. Return the time each person learned the secret.
2. Meetings arrive as a live stream in time order. What changes?
3. Relate it to a real system: spreading a fraud flag or a compromised credential through a network of accounts that transacted together.

Q12. First Occurrence of 1 in a Sorted 0/1 Array

Duration: 15 minutes · **Language:** Java · **Source:** IDFC Technical Round 1, per LeetCode ("interviewer wanted a single-iteration solution")

Problem Statement

You are given an array that contains only `0`s followed by `1`s, so it is sorted. Return the **index of the first `1`**, or `-1` if there are no 1s.

The solution must be a **single binary search loop** that runs in O(log n), even when the array is all 1s. The candidate in the LeetCode report found a 1 and then scanned left from it. That degrades to O(n) when the array is all 1s, and it cost them the round.

```
int firstOne(int[] a)
```

Examples

```
[0,0,0,1,1]  -> 3
[1,1,1,1]    -> 0
[0,0]        -> -1
[]           -> -1
[1]          -> 0
[0]          -> -1
[0,1]        -> 1
```

Points to explain

- The lower-bound pattern: on `a[mid] == 1`, record `ans = mid` and set `hi = mid - 1`. On a 0, set `lo = mid + 1`.
- `mid = lo + (hi - lo) / 2` avoids overflow. Know why `(lo + hi) / 2` overflows for arrays of about 2^31.
- The loop invariant, and why the loop ends.

Follow-ups

1. **Count the 1s:** `n - firstOne` (careful with `-1`).
2. **Unknown length or an infinite stream:** `interface Bits { int get(long i); }` returns `-1` past the end. Use **exponential search**: double `hi` until you find a 1 or the end, then binary search. O(log p), where p is the answer's position.
3. **Matrix:** each row is sorted 0s then 1s. Find the row with the most 1s in O(rows + cols) with a staircase walk.
4. Make it generic: `firstTrue(lo, hi, IntPredicate p)`. The same code solves "first bad version" and "minimum capacity to ship".

Q13. Banking SQL: JOIN, GROUP BY, HAVING

Duration: 25 minutes on paper · **Source:** IDFC Technical Round 1, per LeetCode (a basic JOIN across several tables, then a GROUP BY). The candidate lost marks for using `WHERE` where `HAVING` was needed.

Schema and sample data

```
customers(id, name, city)
  (1,'Asha','Bangalore'), (2,'Bala','Mumbai'), (3,'Chen','Bangalore'), (4,'Dev','Pune')

accounts(id, customer_id, type, opened_on)
  (101,1,'SAVINGS','2024-01-10'), (102,1,'CURRENT','2025-03-05'), (103,2,'SAVINGS','2023-07-21'),
  (104,3,'SAVINGS','2026-02-14'), (105,4,'SAVINGS','2026-09-30')

transactions(id, account_id, txn_type, amount, txn_date)
  (1,101,'CREDIT',50000,'2026-10-01'), (2,101,'DEBIT',12000,'2026-10-03'),
  (3,102,'CREDIT',80000,'2026-10-05'), (4,102,'DEBIT',30000,'2026-10-12'),
  (5,103,'CREDIT',40000,'2026-09-28'), (6,103,'DEBIT',45000,'2026-10-15'),
  (7,104,'CREDIT',20000,'2026-10-02'), (8,104,'DEBIT',5000,'2026-10-20'),
  (9,101,'DEBIT',3000,'2026-10-21')
```

Questions (write the query, then check it against the expected result)

1. For each account, show the customer name, account id and net balance (credits − debits). Accounts with no transactions show 0.

```
Asha 101 35000 | Asha 102 50000 | Bala 103 -5000 | Chen 104 15000 | Dev 105 0
```

Hint: `LEFT JOIN`, `COALESCE`, `SUM(CASE …)`.

2. Total **debit** amount per city in **October 2026**. Show only cities with total debits **> 20000**, highest first.

```
Bangalore 50000 | Mumbai 45000
```

The `txn_type` and date filters go in `WHERE`. The `> 20000` condition goes in `HAVING`.

3. Customers who hold more than one account.

```
Asha 2
```

4. Customers whose accounts have **no transactions at all**.

```
Dev
```

Hint: `LEFT JOIN … WHERE t.id IS NULL`, or `NOT EXISTS`. Be ready to explain the difference from `NOT IN` when there are NULLs.

5. For each city, the customer with the highest total credits. Include ties.

```
Bangalore Asha 130000 | Mumbai Bala 40000
```

Hint: *`RANK() OVER (PARTITION BY city ORDER BY SUM(amount) DESC)`.*

6. Accounts with a negative net balance (overdrawn).

```
103
```

Follow-up talk

- The logical order a query runs in: `FROM/JOIN → WHERE → GROUP BY → HAVING → SELECT → ORDER BY`. This explains why `WHERE` can't use `SUM(...)`.
- Which index helps Q2 (for example `transactions(account_id, txn_date)`), and what `EXPLAIN` should show.
- `INNER` vs `LEFT` join, and what happens to Q1 if you write `WHERE t.txn_type = 'CREDIT'` after a LEFT JOIN.
- Keeping balances consistent: a stored `balance` column updated in the same transaction vs computed on the fly. Isolation levels and `SELECT … FOR UPDATE`.

Q14. AWS: ECS on EC2 vs ECS on Fargate

Duration: 10-minute verbal answer · **Source:** IDFC Technical Round 1, per LeetCode ("why use Fargate over ECS?", asked again after the first answer)

Question

"Your Spring Boot microservice runs on ECS. Why would you choose the **Fargate** launch type over the **EC2** launch type, and when would you not?"

The catch

Fargate isn't an alternative to ECS. It's a **launch type of ECS** (and of EKS). The real comparison is **ECS on EC2** (you manage the instances) against **ECS on Fargate** (serverless compute for containers). Say this first; it's probably why the interviewer asked again.

What a complete answer covers

Dimension	ECS on EC2	ECS on Fargate
Who manages servers	You: AMI, patching, the ECS agent, capacity providers and auto scaling groups	AWS; you only declare task CPU and memory
Scaling	Two layers: tasks *and* instances (cluster auto scaling)	Task-level only
Pricing	Per instance; cheaper at steady, high use and with Spot or Savings Plans	Per vCPU-second and GB-second; a premium for convenience, but Fargate Spot exists
Bin-packing	You can pack many tasks onto one instance	Each task is isolated in its own micro-VM
Control	GPUs, custom kernels, daemon tasks, host volumes, privileged mode	No GPU, no daemons, limited host access
Start time	Fast if capacity is already warm	Image pull on every start. Slower; reduce with smaller images
Security	Shared host; you harden it	Strong task isolation, smaller attack surface

Good answer shape: "For a typical stateless microservice with spiky traffic and a small team, I'd choose Fargate: no instance management, task-level scaling, and better isolation. I'd choose EC2 for steady high use where cost matters, or when I need GPUs, daemon tasks or host-level tuning."

Likely follow-ups

1. Fargate vs **Lambda** for the same service: run length (Lambda's limit is 15 minutes), cold starts, keeping connection pools, cost at high requests per second.
2. How you would auto-scale the service: target tracking on CPU or memory, or on request count per target, through Application Auto Scaling.
3. The scaling policy you used in your own project, and why. (This was asked in another IDFC round.)

4. Where secrets live (Secrets Manager or SSM injected into the task definition) and the IAM task role vs the execution role.

> Relate this to your own AWS experience (Lambda, SQS, Kinesis, IoT Core, API Gateway, CloudWatch). Don't claim hands-on production experience with ECS, Docker or Kubernetes. Say you understand the trade-offs and have worked mostly on the serverless and messaging side.

Q15. Implement the Methods of a Given Class: Bank Account

Duration: 30 minutes · **Language:** Java · **Source:** IDFC online assessment, per LeetCode ("implement methods in a Java class", easy to medium). Bank-themed, like IDFC's domain.

Problem Statement

You are given the class skeleton below. **Don't change the method signatures.** Implement every method so that the rules hold.

```
public enum TxnType { DEPOSIT, WITHDRAWAL, TRANSFER_IN, TRANSFER_OUT, INTEREST }
public record Txn(TxnType type, BigDecimal amount, LocalDate date, BigDecimal balanceAfter) {}

public class Account {
    public static final BigDecimal MIN_BALANCE = new BigDecimal("1000");
    public static final BigDecimal DAILY_WITHDRAWAL_LIMIT = new BigDecimal("50000");
    public static final BigDecimal ANNUAL_RATE = new BigDecimal("0.04");

    public Account(String id, BigDecimal openingBalance, Clock clock) { }  // opening balance >=
MIN_BALANCE
    public BigDecimal balance() { }
    public void deposit(BigDecimal amount) { }
    public void withdraw(BigDecimal amount) { }
    public void transfer(Account to, BigDecimal amount) { }
    public BigDecimal applyMonthlyInterest() { }        // returns the interest credited
    public List<Txn> statement(LocalDate from, LocalDate to) { }  // inclusive, oldest first,
unmodifiable
}
```

Rules

1. Every amount must be **> 0**. Otherwise throw `IllegalArgumentException`.
2. `withdraw` and `transfer` must never leave the balance **below `MIN_BALANCE`**. If they would, throw `InsufficientFundsException`, a custom checked or unchecked exception (your choice; justify it).
3. Withdrawals plus outgoing transfers on the **same calendar day** (from `clock`) can't exceed `DAILY_WITHDRAWAL_LIMIT`. If they would, throw `DailyLimitExceededException`.
4. `transfer` is **atomic**: either both accounts change, or neither does. Transferring to the same account is an `IllegalArgumentException`.
5. Monthly interest = `balance × ANNUAL_RATE / 12`, rounded to 2 decimals, half up.
6. A failed operation must leave **no** partial state: no transaction recorded and no change to the daily total.

Example

```
day 1: A = new Account("A", 10000)
A.deposit(5000)            -> balance 15000
A.withdraw(14500)          -> InsufficientFundsException (would leave 500 < 1000)
A.withdraw(14000)          -> balance 1000

day 1: B = new Account("B", 200000)
B.withdraw(30000)          -> balance 170000
B.withdraw(25000)          -> DailyLimitExceededException (55000 > 50000 today)
day 2: B.withdraw(25000)   -> balance 145000
```

```
day 2: A.transfer(B, 500)  -> InsufficientFundsException; A stays 1000, B stays 145000
day 2: B.transfer(A, 10000)-> B 135000, A 11000   (B's day-2 total is 35000, within the limit)
A.applyMonthlyInterest()   -> 36.67  (11000 x 0.04 / 12 = 36.666...), A = 11036.67
A.statement(day1, day2)    -> DEPOSIT 5000, WITHDRAWAL 14000, TRANSFER_IN 10000, INTEREST 36.67
```

What the panel checks

- `BigDecimal` used with `compareTo`, not `equals`. Know why `new BigDecimal("2.0").equals(new BigDecimal("2.00"))` is false.
- Validation happens **before** any change, which keeps atomicity simple.
- Encapsulation: `statement()` returns an unmodifiable copy, and there's no public setter for the balance.
- `Clock` is injected, so "the same day" can be tested.

Follow-ups

1. **Concurrency:** two threads run `A.transfer(B)` and `B.transfer(A)` at the same time. Show the deadlock, then fix it by **always locking the account with the smaller id first**.
2. Savings vs current accounts: a different minimum balance and no interest. Would you use inheritance or a policy object?
3. Idempotency: a client retries a transfer after a timeout. Add a `requestId` and deduplicate.
4. Move this to Spring Boot with JPA: `@Transactional`, optimistic locking with `@Version`, and what happens on `OptimisticLockException`.